Prediction of Traffic Accident with Machine Learning

Background & Objectives

As one of the major subjects of Data Analytics Course, this Project offers a chance to create own machine learning programme. Picking injurable traffic accident as the theme, the Project initializes with following objectives:

- To practise machine learning (ML) with data analysis on traffic accidents
- To compare the performance of ML models
- To apply the model to predict real-life scenario: risk of injury on a road

Dataset and Basic EDA

Introduction to the dataset

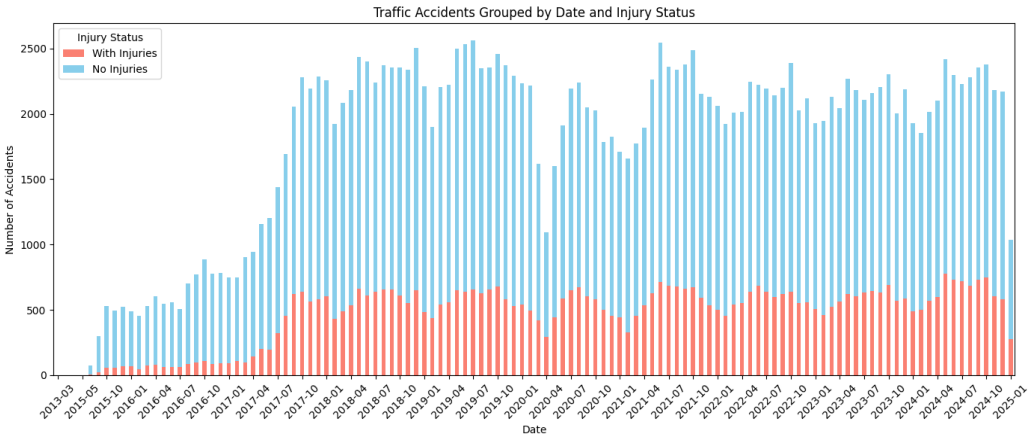
The dataset captures traffic accident records[1] across multiple regions spanning the years 2013 to 2025. It comprises over 200,000 rows and 24 columns. Each entry includes detailed information such as the date of the incident, weather conditions, road surface status, crash type, financial damage incurred, and any resulting injuries and casualties.

EDA and Feature Selection

A basic exploratory data analysis (**EDA**) was carried out for initial understanding to the dataset, attributes and possible relationship. Unilateral, bilateral and multi-lateral analysis were examined before further processing to the data. Below are a few examples.

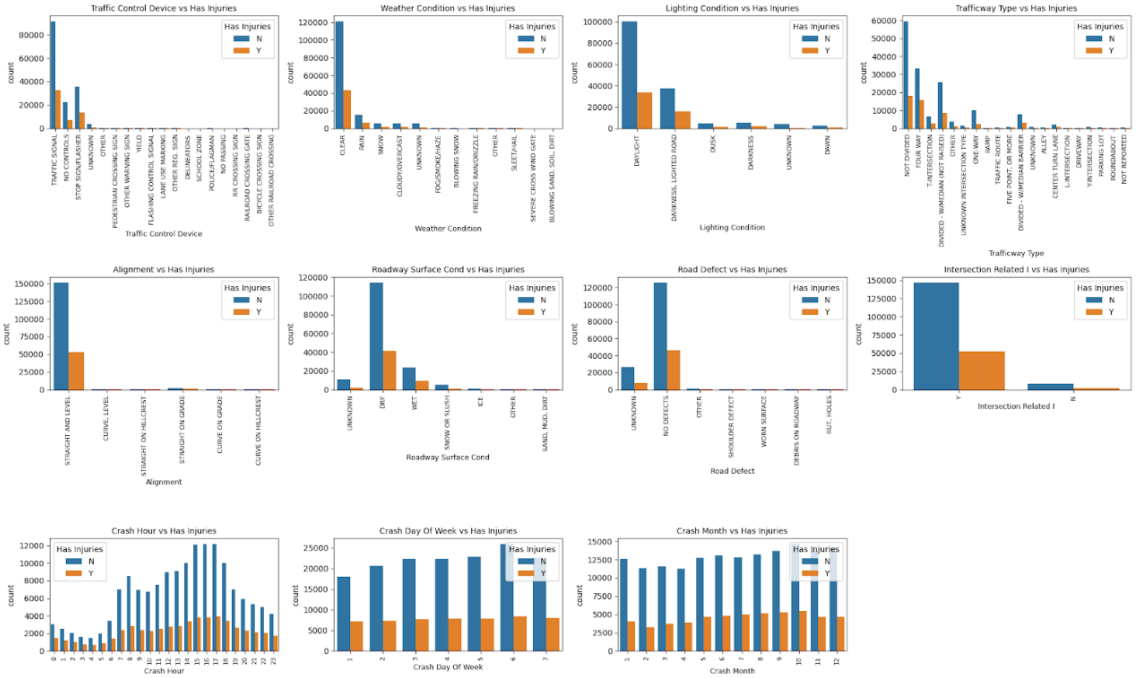
Also, feature importances (App 2) were analyzed and feature selection was applied to examine feature importance scores. The top-ranked features were selected based on these scores, and the model was retrained using only these chosen features. (App 3) This approach reduces noise, accelerates training, enhances model accuracy and generalization, and lowers the risk of overfitting.

Bilateral: The histogram displays traffic accident records spanning from 2013 to 2024. To a certain extent, it shows that season can be a factor to injurable accident (e.g. less injury in every Feb and Mar)



Multi-lateral: The graphs illustrate the relationship between various input features and the injury status in traffic accidents. It facilitates the coming preprocessing for dropping irrelevant data.

[1] Traffic accidents. (2025, January 20). Kaggle. https://www.kaggle.com/datasets/oktayrdeki/traffic-accidents



Unilateral: a simple compilation comparing "has_injuries" or not where 0 represents no injuries and 1 indicates the presence of casualties, reflecting a moderate imbalance of the category in a ratio of about 3:1.

```
Class distribution of "has_injuries":
has_injuries
0    154789
1     54517
Name: count, dtype: int64

y_train class distribution:
has_injuries
0              123958
1               43486
Name: count, dtype: int64

y_test class distribution:
has_injuries
0              30831
1              11031
Name: count, dtype: int64
```

Data Preprocessing and Feature Engineering

After EDA, a clearer strategy for data preprocessing is formed:

1. Categorizing into two groups: cause-related features (e.g., weather conditions, road defects) as input variables x, and result-related features (e.g., injuries, fatalities) as output result y.
2. Eleven columns have been selected as the input features for the study (App 1).
3. Factorizing the string data of x-variables as numerically encoded for model training.
4. Casualty details—including counts of non-incapacitating injuries, incapacitating injuries, and fatalities—are consolidated into a single binary output feature, has_injuries, where 0 represents no injuries and 1 indicates the presence of casualties in traffic accident records.
5. Splitting into a training set (80%) and a test set (20%).
6. Given the previously noted class imbalance, resampling is applied to the training set to assess its impact on model performance and explore potential improvements.

Machine Learning Methodology

Genre

Supervised learning, used in our study, involves training algorithms on labeled data, where each data point has a known outcome or label. The algorithm learns to predict the correct output for new data by recognizing patterns from these labeled examples.

Rationale: Injury is a very serious outcome. It should not be vague and wrongly recognized by the algorithm. Otherwise, this Study may not only yield the right prediction, but also wrong indication that would result in real injury and casualties.

Models

Logistic Regression predicts the probability of binary or multiclass outcomes by applying a sigmoid function to a linear combination of features and classifies data using a threshold. It is a classification approach log-odd calculation to predict a categorial result (has injury / no injury, high risk / low risk)

Rationale - accuracy: This study requires more on classification type rather than clustering type of machine learning when analyzing the attributes to injury. Also, it outputs a categorial prediction (yes / no) rather than a linear prediction (1, 0.2, 3.4...). Most importantly, its prediction method is a "maximum likelihood estimation" which is good for this Study use on injury prediction. In addition, it has potential to expand from binomial outcome (yes / no) to ordinal result (low / mid / high risk).

Random Forest is an ensemble method that creates multiple decision trees from random subsets of data and features, with the final prediction based on majority voting.(App 4)

Rationale - less overfitting and clear key factor identification: Random Forest handles complex, large, and noisy data well (e.g. day / week / month). It is better than "single" Decision Tree by reducing overfitting by averaging many trees, works without much data preprocessing, and can manage missing values. It also ranks feature importance, helping identify key factors, like crash time, traffic and weather conditions. This helps reduce overfitting and captures complex nonlinear relationships.

Histogram-based Gradient Boosting Classification Tree (HGB) builds trees sequentially, each correcting the errors of the previous one, and uses histograms to speed up training and reduce memory usage, improving accuracy on complex data.
Rationale - speed: The Dataset of this Study includes 200,000 records with 25 attributes. Relatively the size is large. HGB, by using histogram and estimate split points, consumes relatively less computation and runs the result in a shorter time.

Overfitting / Underfitting Detection

Stratified **Cross-validation** with the **F1 score** is employed to maintain class balance across folds. The model is optimized by focusing on the macro F1 score to ensure balanced performance across classes. The best cross-validation score is the one that indicates strong generalization and helps reduce overfitting, not just the highest number.

In addition, **Randomized Search** adjusts **class weights** during hyperparameter tuning to give more importance to minority classes. This helps the model better handle imbalanced data by penalizing misclassification of minority classes more heavily while exploring hyperparameters randomly. This technique improves minority class performance within the randomized search process.

Thirdly, **Bayesian Optimization** also serves as a "proof by contradiction" to show there is no obvious overfitting / underfitting problem. By **"C-value" and "penalty: l2"**, there is no significant change on the result, proving no overfitting / underfitting issue.[2]

Tuning Method

Resampling: Class Weight Balance and SMOTE

During model tuning, function "class_weight = balanced" is applied to try to solve the imbalance distribution of "has_injuries". Alternatively, SMOTE is another library imported for the same purpose.

Hyperparameter Tuning

Randomized Search is a method in iteration with a random combination of attributes and result to look for the best performing set. Prior to adopting randomized search for hyperparameter tuning in this study, grid search was initially employed. However, it proved to be computationally expensive and inefficient (30 mins). In contrast, randomized search samples a fixed number of parameter combinations, offering a more time-efficient approach (< 3 mins) while still effectively exploring the search space.
Rationale: A widely used tuning method with proven improved results in a more timely manner.

Bayesian Optimization, unlike Grid Search and Randomized Search that try all or blindly select combinations, applies surrogate function to model the relationship between hyperparameters and the score. In short, it learns immediately from past result to determine the next combination to try, until an optimal set is found.
Rationale: While Grid Search is consuming and Randomized Search works like a blind, Bayesian took the balance between the two. Where Randomized Search did not show significant improvement, this method appears as an alternative hyperparameter tuning way for this Study.

Analysis and Evaluation on Models

Metrics

- Accuracy: Measures overall correctness of predictions
- F1 score: Examine the model's ability to correctly identify both injury and no-injury cases
- F2 score: How effectively the model identifies actual injury cases
- Confusion matrix: compare the distribution of true positive, true negative, false positive and false negative prediction

Performance of Models

[2] Rithp. (2023, January 5). Logistic regression and regularization: Avoiding overfitting and improving generalization. Medium. https://medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d

Model 1: Logistic Regression

Approach 1: Train with the originally split data

It achieved an accurate rate at 73.65%, with F1 and F2 scores at 0.42 and 0.00. From the confusion matrix, it shows that the model makes all prediction to "no injuries". It reflects a big defect of the model in outputting a skewed result. (App 5.1)

Approach 2: Train with resampled data

Function parameter "class_weight=balanced" and SMOTE method are used to solve the 1:3 imbalance distribution. Accuracy drops significantly to 46 to 48%, but a rise of F1 and F2 scores to around 0.47 and 0.50. Confusion Matrix also shows a more balanced prediction. Due to unacceptable accuracy, it is decided to go for hyperparameter tuning for Logistic Regressing based on original imbalance data. (App 5.2, 5.3)

Approach 3: Hyperparameter Tuning using Randomized Search

On account of the nature of Logistic Regression algorithm in log-odd and sigmoid function, Randomized Search made almost no change to the performance. Another tuning method should be tried. (App 5.4)

Approach 4: Hyperparameter Tuning using Bayesian Optimization

By using 'C' value and penalty L2, the ultimate result with best score is about 0.7403. Although it sounds insignificant, a 0.3% improvement in a size of 200,000 records actually means 600 more correct predictions.

Model 2 - Random Forest

Approach 1: Train with the originally split data

Random Forest trained on the original data achieved 68.67% accuracy, with an F1 score of 0.49 and a low F2 score of 0.15, indicating moderate overall performance but poor emphasis on recall. (App 5.5)

Approach 2: Hyperparameter Tuning using Randomized Search

Accuracy improved to 73.6%, but F1 dropped to 0.42 and F2 remained very low. This means the balance between precision and recall worsened, so although overall accuracy increased, the model's ability to handle imbalanced data may have weakened. (App 5.6)

Approach 3: Apply Cross-validation Technique

The results show the best cross-validated F1 score is about 0.48, and the test set accuracy is 71.20% with an F1 score of 0.48. Evaluation on separate test data gives an unbiased performance estimate. This shows a reliable tuning process improving F1 score and handling class imbalance well. The test accuracy and F1 are consistent with cross-validation, indicating good generalization. Overall, this tuning is valid because it stabilizes estimation, optimizes F1—which is better for imbalance compared to accuracy—checks overfitting, and balances model complexity with generalization. (App 5.7)

Approach 4: Apply Class Weight Balancing Technique

After applying the parameter class_weight=balanced, although accuracy drops to 60%, the F1 and F2 scores improve significantly to 0.54, demonstrating a better balance between precision and recall and notably enhancing performance on minority classes. (App 5.8)

Approach 5: Hyperparameter Tuning using Bayesian Optimization

Test accuracy slightly dropped to 66.33%, but F1 and F2 scores improved to 0.50, showing better balance between precision and recall. Compared to other methods, this approach achieves better tuning results faster, especially on imbalanced datasets. (App 5.9)

Model 3: Histogram-based Gradient Boosting Classification Tree

Approach 1: Train with the originally split data

The model achieved an accuracy of 73.69%, an F1 score of 0.427, and an F2 score of 0.00 on the test set. Based on the confusion matrix, the model demonstrated strong performance in predicting non-injury cases but failed to identify instances involving casualties. (App 5.10)

Approach 2: Hyperparameter Tuning using Randomized Search

Using the optimal parameters identified through randomized search, the model's performance remained nearly unchanged compared to the default configuration. It yielded 73.66% accuracy, an F1 score of 0.428, and an F2 score of 0.00, indicating that hyperparameter tuning alone did not improve the model's ability to detect injury cases.

Approach 3: Train with resampled dataset

Significant improvement was observed when the model was trained on a resampled dataset to address class imbalance. The model achieved an F1 score of 0.521 and an F2 score of 0.48, reflecting enhanced sensitivity to casualty predictions. Although overall accuracy dropped to 55.04%, the model gained the critical ability to identify injury-related traffic accidents. (App 5.11)

Model Comparison

Classification reports were generated to offer a detailed breakdown of a model's accuracy for each class, presenting metrics such as precision, recall, F1-score, and support. (App 6) The report typically includes an overall accuracy score, reflecting the total percentage of correct predictions across all classes. This report provides a thorough view of the model's performance, which is especially useful for imbalanced datasets where overall accuracy alone can be misleading.

Accuracy

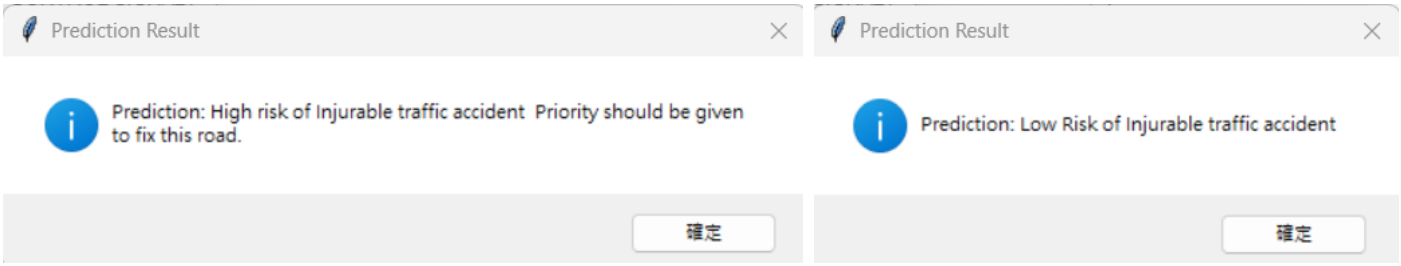
Logistic Regression has the highest accuracy among the three models but extreme confusion matrix. All three models achieve similar best accuracy after hyperparameter tuning, with greatest improvement on Random Forest.

F1 and F2 scores
Resampling, Randomized Search, and class weight balancing boost F1 and F2 scores across models but significantly reduce accuracy, indicating a trade-off to focus on minority class performance and force rebalance of prediction distribution.
Best performer
Considering all 3 metrics, unlike the other 2 models result in tradeoff between metrics, **Random Forest** achieves similar accuracy while keeping steadily good F1 and F2 scores after tuning, making it the best performer. See App 7 for a general comparison.

Challenges, Limitations and Future Work

Challenges and Limitations	Future Improvements
Class imbalance in test set	Better way of resampling Further refine the models (e.g. use precision-recall Curve to maximize the recall)
Noisy features (e.g. alignment, intersection_related_i) hinders the performance	Deeper EDA and revisit the feature importance of each attributes for future preprocessing
Too vague to only output 0 (no injuries) and 1 (has injury)	Turn to ordinal prediction (high / mid / low risk), or even a generated probability
Too general for “has_injuries” output only	Break down to mild, incapacitating, fatal injuries prediction

Application
This Project not only explores the use of machine learning but also aims at applying the trained model to real life application. Therefore, a simple GUI is compiled. It allows users to enter attributes based on real scenarios, to predict the tendency / risk level (high or low) of an injurable traffic accident that may happen. See App 8 for screen cap and .ipynb notebook for trial.



More importantly, the prediction result is traceable and explainable in terms of feature importance and its coefficient (see App 2).

Conclusion & Insight
This Project selects 3 machine learning models for training to predict injurable traffic accidents based on preprocessed data attributes. Imbalance distribution of injuries (y) in 1:3 remains the biggest challenge. It is encouraging that all models still give an accuracy to about 75%. After resampling, overfitting reduction and hyperparameter tuning,Random Forest is found to be the optimal model for not only accuracy but also balance in prediction of both binary results.

One big insight is that machines can never like human to understand the meaning and value of life. Some models give a high accuracy result by extremely skewing the prediction to one side. In human sense, injury and risk is not a logic calculation of accuracy. Human would prefer warning even it is under 50% or sounds “low” risk instead of simply prompting “low injury risk”. Nevertheless, it can improve by showing probability rate, and still be applicable for use case like prioritizing road defect fixing schedule.

<END>

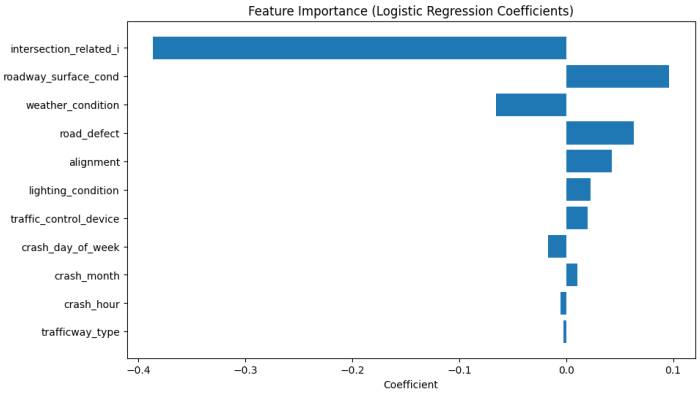
Appendix

App 1: Features of the dataset

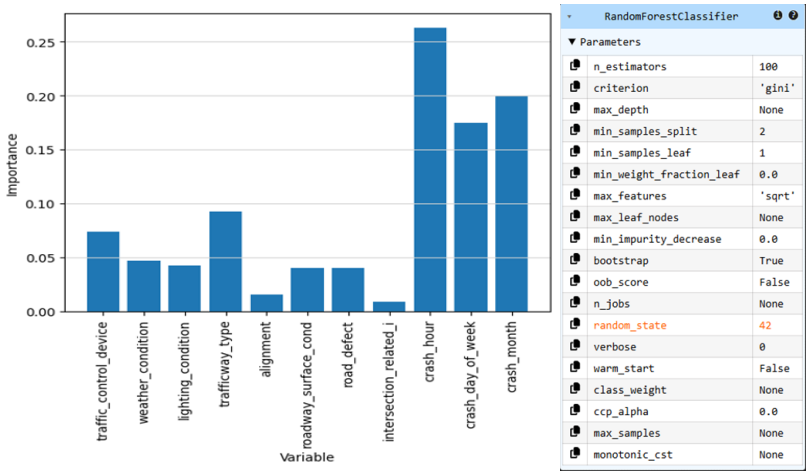
No.	Feature	Description	Classification	Type
1	traffic_control_device	The type of traffic control device involved (e.g., traffic light, sign)	Categorical	Nominal
2	weather_condition	The weather conditions at the time of the accident.	Categorical	Nominal
3	lighting_condition	The lighting conditions at the time of the accident.	Categorical	Nominal
4	trafficway_type	The type of roadway involved in the accident (e.g., highway, local road).	Categorical	Nominal
5	alignment	The alignment of the road where the accident occurred (e.g., straight, curved).	Categorical	Nominal
6	roadway_surface_cond	The condition of the roadway surface (e.g., dry, wet, icy).	Categorical	Nominal
7	road_defect	The condition of the roadway surface (e.g. no defects, worn surface).	Categorical	Nominal
8	intersection_related_i	Whether the accident was related to an intersection.	Categorical	Nominal
9	crash_hour	The hour the accident occurred.	Numerical	Discrete
10	crash_day_of_week	The day of the week the accident occurred.	Numerical	Discrete
11	crash_month	The day of the week the accident occurred.	Numerical	Discrete

App 2: Feature importances

(1) Logistic Regression feature importances (Coefficients)



(2) Random Forest feature importances



RandomForestClassifier

Parameters

n_estimators	100
criterion	'gini'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

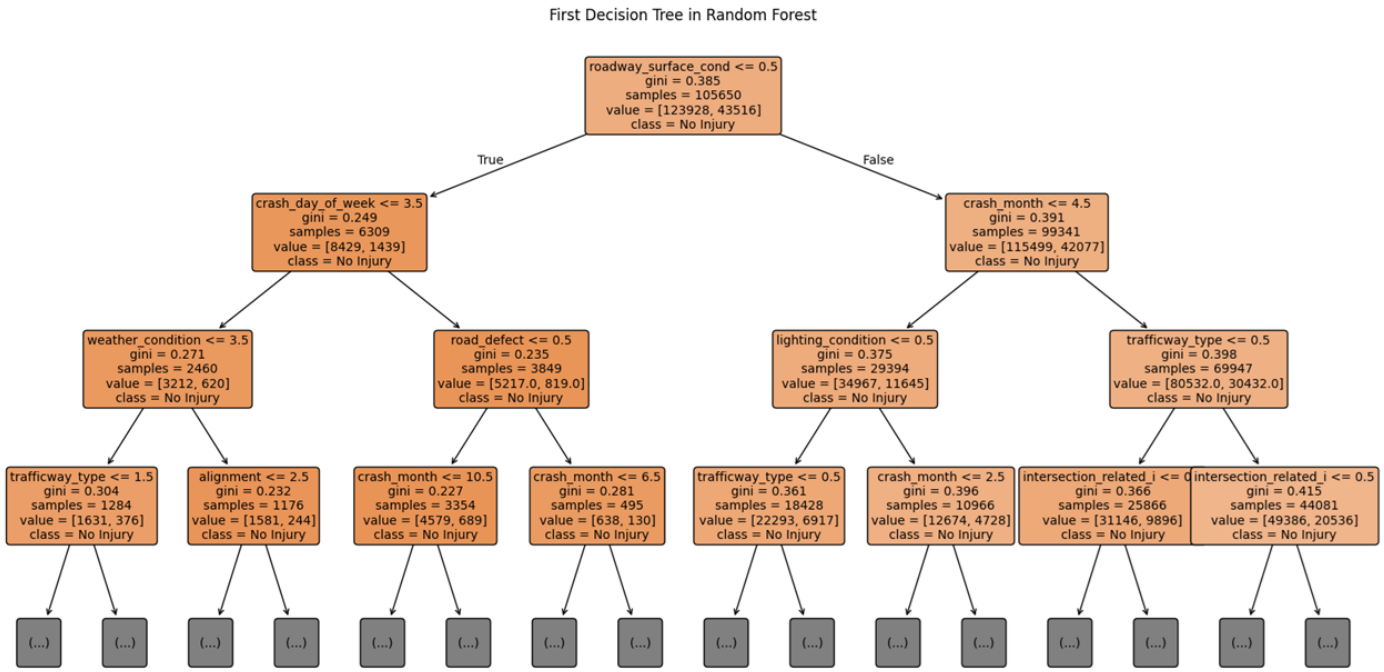
(3) Histogram-based Gradient Boosting feature importances (Permutation)

```
Feature Importances (Permutation):
traffic_control_device: 0.0011 +/- 0.0001
trafficway_type: 0.0003 +/- 0.0000
crash_hour: 0.0003 +/- 0.0000
crash_month: 0.0003 +/- 0.0000
lighting_condition: 0.0002 +/- 0.0000
roadway_surface_cond: 0.0002 +/- 0.0000
crash_day_of_week: 0.0002 +/- 0.0000
alignment: 0.0001 +/- 0.0000
weather_condition: 0.0001 +/- 0.0000
road_defect: 0.0001 +/- 0.0000
intersection_related_i: 0.0000 +/- 0.0000
```

App 3: Selected features in Feature Selection

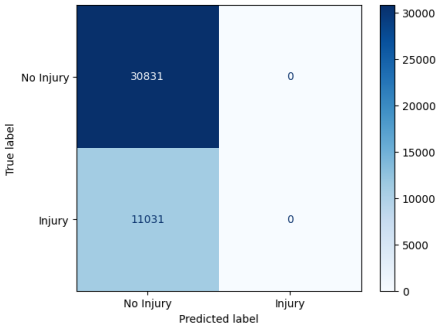
```
Selected features:
 ['crash_hour', 'crash_month', 'crash_day_of_week', 'trafficway_type', 'traffic_control_device', 'weather_condition', 'lighting_condition', 'road_defect', 'roadway_surface_cond', 'alignment']
```

App 4: First Decision Tree in Random Forest

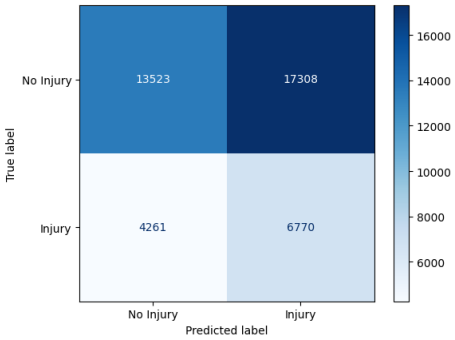


App 5: Confusion Matrix

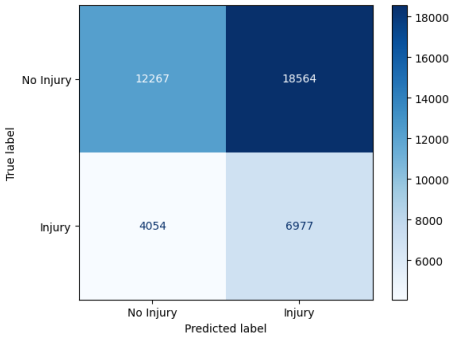
(1) Confusion Matrix: Linear Regression Model Trained on Original Data Split



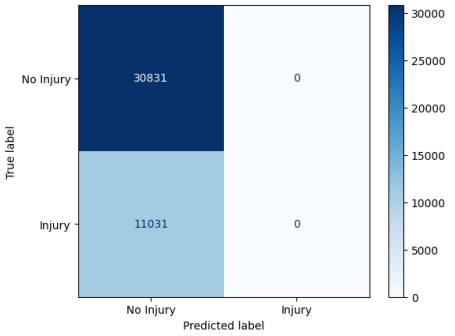
(2) Confusion Matrix: Linear Regression Model with “class_weight=balanced” Applied



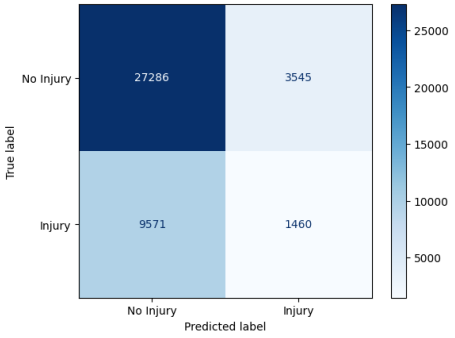
(3) Confusion Matrix: Linear Regression Model Trained on Resampled Data using SMOTE



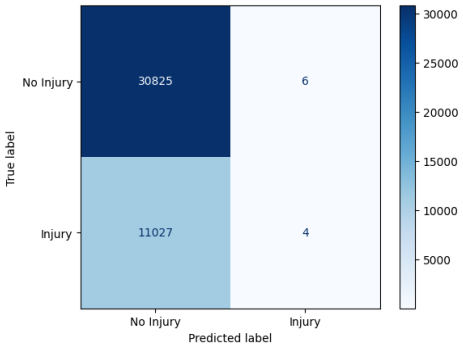
(4) Confusion Matrix: Linear Regression Model with Hyperparameter Tuning via Randomized Search



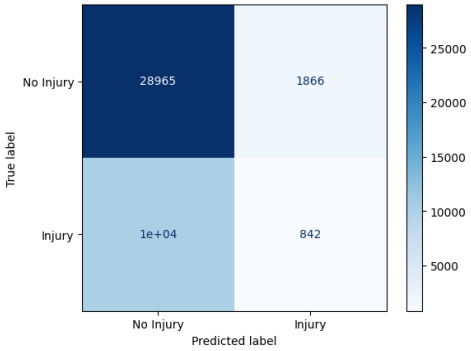
(5) Confusion Matrix: Random Forest Model Trained on Original Data Split



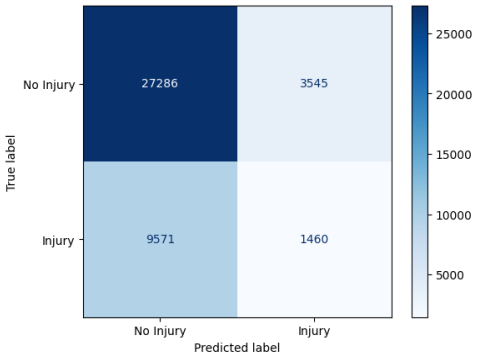
(6) Confusion Matrix: Random Forest Model with Hyperparameter Tuning via Randomized Search



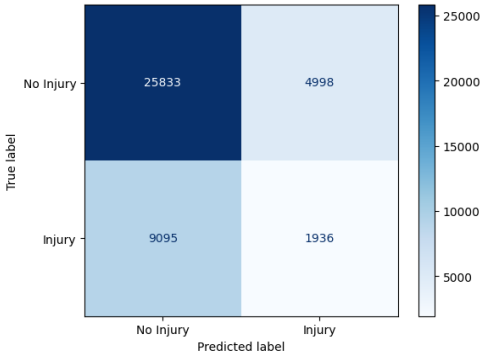
(7) Confusion Matrix: Random Forest Model with Cross-validation Applied



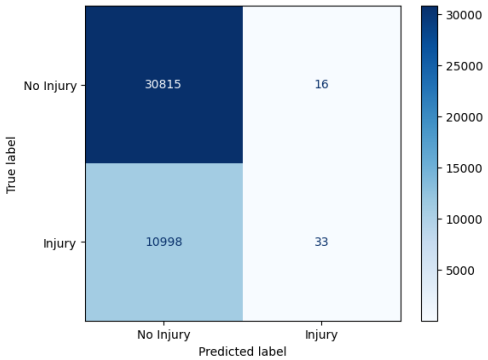
(8) Confusion Matrix: Random Forest Model with “class_weight=balanced” Applied



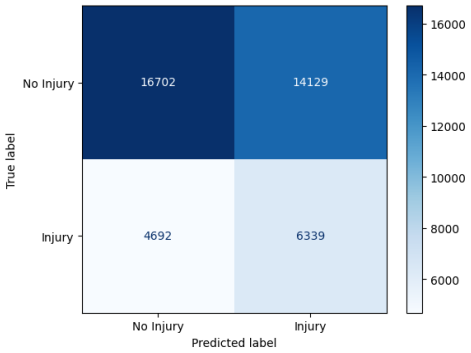
(9) Confusion Matrix: Random Forest Model with Hyperparameter Tuning via Bayesian Optimization



(10) Confusion Matrix: Histogram-based Gradient Boosting Model Trained on Original Data Split



(11) Confusion Matrix: Histogram-based Gradient Boosting Model Trained on Resampled Data using SMOTE



App 6: Classification Reports

(1) Logistic Regression Classification Report

Logistic Regression Classification Report
Originally Training Dataset:

	precision	recall	f1-score	support
0	0.74	1.00	0.85	30831
1	0.00	0.00	0.00	11031
accuracy			0.74	41862
macro avg	0.37	0.50	0.42	41862
weighted avg	0.54	0.74	0.62	41862

With Class_weight Balanced:

	precision	recall	f1-score	support
0	0.76	0.44	0.56	30831
1	0.28	0.61	0.39	11031
accuracy			0.48	41862
macro avg	0.52	0.53	0.47	41862
weighted avg	0.63	0.48	0.51	41862

After Resampling:

	precision	recall	f1-score	support
0	0.75	0.40	0.52	30831
1	0.27	0.63	0.38	11031
accuracy			0.46	41862
macro avg	0.51	0.52	0.45	41862
weighted avg	0.63	0.46	0.48	41862

After RandomizedSearch:

	precision	recall	f1-score	support
0	0.74	1.00	0.85	30831
1	0.00	0.00	0.00	11031
accuracy			0.74	41862
macro avg	0.37	0.50	0.42	41862
weighted avg	0.54	0.74	0.62	41862

(2) Random Forest Classification Report

Random Forest Classification Report
Originally Training Dataset:

	precision	recall	f1-score	support
0	0.86	0.97	0.91	30831
1	0.86	0.56	0.68	11031
accuracy			0.86	41862
macro avg	0.86	0.76	0.79	41862
weighted avg	0.86	0.86	0.85	41862

After RandomizedSearch:

	precision	recall	f1-score	support
0	0.74	1.00	0.85	30831
1	0.40	0.00	0.00	11031
accuracy			0.74	41862
macro avg	0.57	0.50	0.42	41862
weighted avg	0.65	0.74	0.62	41862

After Cross-validation:

	precision	recall	f1-score	support
0	0.74	0.94	0.83	30831
1	0.31	0.08	0.12	11031
accuracy			0.71	41862
macro avg	0.53	0.51	0.48	41862
weighted avg	0.63	0.71	0.64	41862

with Class_weight balanced:

	precision	recall	f1-score	support
0	0.77	0.66	0.71	30831
1	0.32	0.44	0.37	11031
accuracy			0.60	41862
macro avg	0.54	0.55	0.54	41862
weighted avg	0.65	0.60	0.62	41862

After Bayesian Optimization:

	precision	recall	f1-score	support
0	0.74	0.84	0.79	30831
1	0.28	0.18	0.22	11031
accuracy			0.66	41862
macro avg	0.51	0.51	0.50	41862
weighted avg	0.62	0.66	0.64	41862

(3) Histogram-based Gradient Boosting Classification Report

```
Histogram-based Gradient Boosting Classification Report
Originally Training Dataset:
              precision    recall  f1-score   support

           0       0.74      1.00      0.85     30831
           1       0.62      0.00      0.01     11031

    accuracy                           0.74     41862
   macro avg       0.68      0.50      0.43     41862
weighted avg       0.71      0.74      0.63     41862

After RandomizedSearch:
              precision    recall  f1-score   support

           0       0.74      1.00      0.85     30831
           1       0.52      0.00      0.01     11031

    accuracy                           0.74     41862
   macro avg       0.63      0.50      0.43     41862
weighted avg       0.68      0.74      0.63     41862

After Resampling:
              precision    recall  f1-score   support

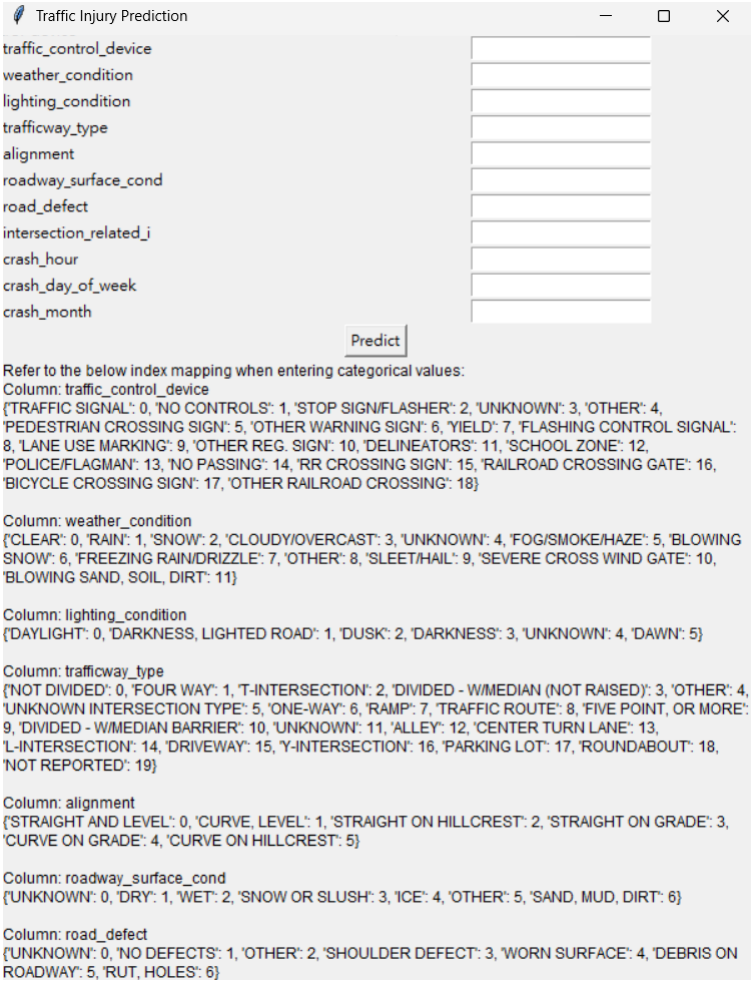
           0       0.77      0.53      0.63     30831
           1       0.30      0.57      0.39     11031

    accuracy                           0.54     41862
   macro avg       0.54      0.55      0.51     41862
weighted avg       0.65      0.54      0.57     41862
```

App 7: Comparison table of Model Performance

Model	Methods	Accuracy	F1 Score	F2 Score
1. Histogram-based Gradient Boosting Classification Tree	Originally training dataset	73.65%	0.43	0.00
	Randomized search	73.66%	0.43	0.00
	Resampling	53.92%	0.51	0.48
2. Logistic Regression	Originally training data	73.65%	0.42	0.00
	Resampling	48.48%	0.47	0.50
	Randomized search	73.65%	0.42	0.00
	Bayesian Optimization (Best score)	74.03%	/	/
3. Random Forest	Originally training dataset	68.67%	0.49	0.15
	Randomized search	73.64%	0.42	0.50
	Cross-validation	71.20%	0.48	0.49
	Resampling with class_weight balanced	60.7%	0.54	0.54
	Bayesian Optimization	66.33%	0.50	0.50

App 8: Real application – A GUI of the trained model for injurable traffic accident prediction



Reference

Traffic accidents. (2025, January 20). Kaggle. <https://www.kaggle.com/datasets/oktayrdeki/traffic-accidents>, last accessed 20 Sept 2025

Rithp. (2023, January 5). Logistic regression and regularization: Avoiding overfitting and improving generalization. Medium. <https://medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d>, last accessed 20 Sept 2025